Virtualization & Docker

Beginner-Friendly Study Material • Concepts, Architecture, Commands, Examples & Viva

Virtualization
A way to create virtual versions of computing resources, such as servers, operating systems, storage, or networks.

Docker
A platform for building, packaging, and running applications in lightweight, isolated containers.

For college study, lab practice, cloud computing and DevOps fundamentals.

1. Learning Objectives

• Define virtualization and explain why it is used.

• Differentiate Type 1 and Type 2 hypervisors.

• Understand virtual machines (VMs) and containers.

• Explain Docker images, containers, registries, volumes and networks.

• Run basic Docker commands and build a simple image using a Dockerfile.

• Understand common use cases, benefits, limitations and security practices.

2. What is Virtualization?

Virtualization is a technology that uses software to create virtual computing environments from physical hardware. One physical machine can host multiple virtual machines (VMs), each with its own operating system and allocated CPU, memory, storage, and network resources.

Main terms

• **Host machine:** physical computer running virtualization software.

• **Guest machine:** virtual machine running on the host.

• **Hypervisor:** software that creates and manages virtual machines.

• **VM:** isolated virtual computer with its own guest operating system.

• **Resource allocation:** assigning CPU, RAM, disk, and network capacity to VMs.

Why use virtualization?

• Run different operating systems on one physical server.

• Improve hardware utilization and consolidate servers.

• Create isolated test and development environments.

• Support snapshots, backups, migration, and disaster recovery.

• Scale infrastructure more flexibly than using one physical server per application.

3. Types of Virtualization

Server / hardware virtualization

A hypervisor divides a physical server into multiple virtual machines.

Desktop virtualization

A desktop environment is hosted centrally or delivered virtually to a user's device.

Storage virtualization

Multiple storage devices are presented as a logical storage resource.

Network virtualization

Software-defined logical networks are created over physical network infrastructure.

Application virtualization

An application is isolated from some details of the underlying OS or delivered without a traditional full installation.

4. Hypervisors: Type 1 vs Type 2

Feature	Type 1 (bare-metal)	Type 2 (hosted)
Where it runs	Directly on server hardware	On top of a host OS

Feature	Type 1 (bare-metal)	Type 2 (hosted)
Typical use	Data centres, enterprise servers	Desktop labs, personal testing
Examples	VMware ESXi, Microsoft Hyper-V, Xen	Oracle VirtualBox, VMware Workstation
General trade-off	Often suited to server-scale management	Convenient for desktop use; has host OS overhead

Product features and terminology can vary by version and configuration.

5. Virtual Machines vs Containers

A VM virtualizes hardware and typically includes a full guest operating system. A container packages an application and its dependencies while sharing the host operating-system kernel through container-runtime isolation.

Feature	Virtual Machine	Container
Operating system	Usually includes a full guest OS	Shares host kernel
Typical footprint	Larger	Usually smaller
Startup	Often slower	Often faster
Isolation boundary	Virtual hardware + hypervisor	OS-level isolation; depends on configuration
Best fit	Different OS kernels, VM separation, legacy workloads	Portable app packaging, microservices, repeatable deplo

Simple analogy

A VM is like a separate house with its own utilities; containers are more like separate apartments sharing the building's underlying infrastructure. The analogy is simplified: containers provide isolation, but not the same boundary as a separate VM.

6. What is Docker?

Docker is a platform and toolset for developing, shipping, and running applications in containers. It helps package an application with libraries and configuration so it can run more consistently across compatible environments.

Core Docker components

- **Docker Engine:** software used to build and run containers.
- **Docker CLI:** command-line interface used to issue Docker commands.
- **Image:** read-only template used to create containers.
- **Container:** a running or stopped instance created from an image.
- **Dockerfile:** text file containing instructions to build an image.
- **Registry:** service that stores and distributes images, such as Docker Hub.
- **Volume:** persistent storage managed by Docker.
- **Network:** lets containers communicate with each other and external systems.

7. Docker Architecture

- **Client:** sends commands such as docker build and docker run.
- **Docker daemon:** manages Docker objects and performs build/run tasks.
- **REST API:** communication interface used by clients and tools.
- **Registry:** stores images that can be pulled to a machine.

On some platforms, Docker Desktop provides a convenient application environment; on Linux, Docker Engine commonly runs as a system service.

8. Install and Verify Docker

Install Docker Desktop on supported Windows or macOS systems, or install Docker Engine using official instructions for your Linux distribution. On Windows, Docker Desktop commonly uses WSL 2 or Hyper-V depending on configuration.

```
docker --version
docker version
docker info
docker run hello-world
```

The hello-world command downloads a small test image if needed and runs it. Successful output indicates Docker can run a basic container.

9. Essential Docker Commands

Command	Purpose
docker pull nginx	Download the nginx image
docker images	List local images
docker run hello-world	Create and run a test container
docker run -d -p 8080:80 --name web nginx	Run Nginx in background and map host port 8080 to container port 8
docker ps	List running containers
docker ps -a	List all containers, including stopped ones
docker logs web	View container logs
docker stop web	Stop a running container
docker start web	Start a stopped container
docker rm web	Remove a stopped container
docker rmi nginx	Remove an image when no longer in use
docker exec -it web sh	Open a shell in a running container, if available

In -p 8080:80, 8080 is the host port and 80 is the container port. Open http://localhost:8080 while the container is running.

10. Build a Simple Docker Image

Create a folder containing index.html and Dockerfile.

index.html

```
<!doctype html>
<html>
  <head><title>Docker Demo</title></head>
  <body>
    <h1>Hello from Docker!</h1>
    <p>This page is served by an Nginx container.</p>
  </body>
</html>
```

Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
```

Build and run

```
docker build -t student-web:1.0 .
docker run -d --name student-web -p 8080:80 student-web:1.0
docker ps
```

Open http://localhost:8080. The final dot in docker build -t student-web:1.0 . means use the current directory as the build context.

```
docker stop student-web
docker rm student-web
```

11. Docker Volumes and Bind Mounts

A container's writable layer is not the best place for important persistent data. Volumes and bind mounts let data exist outside that layer.

• **Named volume:** managed by Docker; useful for persistent application data.

• **Bind mount:** maps a specific host path into a container; useful for development and sharing files.

```
docker volume create app-data
docker volume ls
docker volume inspect app-data
```

12. Docker Networking

• **Bridge network:** common default for containers on a single Docker host.

• **Host networking:** container shares the host network namespace on supported systems.

• **None:** disables normal network connectivity for the container.

• **User-defined bridge:** lets containers on that network communicate by name in common Docker setups.

```
docker network ls
docker network create app-net
docker run -d --name web1 --network app-net nginx
docker network inspect app-net
```

For a web app and database, use an appropriate private network and configure the app to connect to the database by service/container name and port. Avoid exposing database ports publicly unless necessary and secured.

13. Docker Compose

Docker Compose describes and runs multi-container applications using a YAML file.

```
# compose.yaml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"

docker compose up -d
docker compose ps
docker compose logs
docker compose down
```

Run commands in the directory containing compose.yaml.

14. Benefits and Limitations

Benefits

• Consistent environments from development to deployment.

• Efficient resource use compared with many full VMs.

• Fast creation and removal of application environments.

• Supports CI/CD pipelines and microservice architectures.

• Images can be versioned and distributed through registries.

Limitations / considerations

• Containers share the host kernel, so they are not identical to full VMs.

• Persistent storage and backups need deliberate planning.

• Networking, permissions, and image vulnerabilities require security attention.

• Stateful workloads may need careful orchestration and monitoring.

• Containerization does not automatically make an application scalable or secure.

15. Security Best Practices

• Use trusted, maintained base images and update them regularly.

• Avoid running as root inside a container when not needed.

• Do not bake passwords, API keys, or tokens into images.

• Use least-privilege permissions and restrict exposed ports.

• Scan images for known vulnerabilities and pin versions where appropriate.

• Set resource limits and monitor logs and container health.

• Treat containers as isolated but not invulnerable; update host and runtime software.

16. Common Problems and Fixes

• **docker is not recognized / command not found:** install Docker and ensure its CLI is available on PATH.

• **Cannot connect to Docker daemon:** check whether Docker Engine/Desktop is running and the user has required permissions.

• **Port is already allocated:** choose another host port, such as -p 8081:80.

• **Container exits immediately:** inspect docker ps -a and docker logs CONTAINER_NAME.

• **Image pull fails:** check connectivity, image name, tag, and registry access.

• **Changes don't appear:** rebuild the image if the changed file is copied during build; recreate the container if needed.

17. Use Cases

• Cloud infrastructure: run VMs for different operating systems and security boundaries.

• Development: package apps and dependencies into repeatable containers.

• Testing: create disposable environments for automated tests.

• Deployment: deliver the same versioned image across environments.

• Microservices: deploy and scale services independently when supported by suitable infrastructure.

• Education: practice Linux commands, networking, deployment, and DevOps workflows.

18. Quick Revision

Question	Short answer
What is virtualization?	Creating virtual computing resources using software.
What is a hypervisor?	Software that creates and manages virtual machines.
Type 1 hypervisor?	Runs directly on physical hardware.
What is a container?	An isolated application environment sharing the host kernel.
What is Docker?	A platform for building and running containers.
Image vs container?	An image is a template; a container is an instance created from it.
What is a Dockerfile?	Instructions for building an image.
What is a registry?	A service that stores and distributes images.
What is a volume?	Persistent storage managed separately from a container's writable layer.
What does -p 8080:80 mean?	Map host port 8080 to container port 80.

19. Viva / Exam Questions

• Define virtualization and mention four benefits.

• Differentiate Type 1 and Type 2 hypervisors.

• Compare virtual machines and containers.

• Define Docker image and Docker container.

• Explain Docker architecture and its main components.

• What is the purpose of a Dockerfile?

• Explain docker pull, docker run, docker ps, docker stop, and docker rm.

• What is port mapping in Docker?

• Differentiate volumes and bind mounts.

• What is Docker Compose?

• List five Docker security best practices.

• Explain how Docker helps in application deployment.

20. Practical Tasks

• Install Docker and run docker run hello-world.

• Run Nginx on host port 8080 and open it in a browser.

• Build a custom image from the Dockerfile in this PDF.

• Create and inspect a named volume.

• Create a user-defined network and run containers on it.

• Write a Compose file for a simple service and use docker compose up -d.

Official Documentation

• Docker Docs: https://docs.docker.com/

• Docker Get Started: https://docs.docker.com/get-started/

• Microsoft Learn — Virtualization: https://learn.microsoft.com/en-us/virtualization/

These notes are introductory. Installation steps and commands can vary by operating system and Docker version; use official documentation for current platform-specific instructions.